

```
!pip install pandas requests beautifulsoup4 lxml plotly gradio
```

```
Requirement already satisfied: lxml in /usr/local/lib/python3.12/dist-packages (5.0.0)
Requirement already satisfied: plotly in /usr/local/lib/python3.12/dist-packages (5.22.0)
Requirement already satisfied: gradio in /usr/local/lib/python3.12/dist-packages (0.0.1)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (2024.1)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (2024.1)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (2025.1.1)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages (2.6)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.12/dist-packages (4.12.2)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.12/dist-packages (9.0.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (24.2)
Requirement already satisfied: aiofiles<25.0,>=22.0 in /usr/local/lib/python3.12/dist-packages (24.1.2)
Requirement already satisfied: anyio<5.0,>=3.0 in /usr/local/lib/python3.12/dist-packages (4.7.0)
Requirement already satisfied: brotli>=1.1.0 in /usr/local/lib/python3.12/dist-packages (1.1.0)
Requirement already satisfied: fastapi<1.0,>=0.115.2 in /usr/local/lib/python3.12/dist-packages (0.115.2)
Requirement already satisfied: ffmpeg in /usr/local/lib/python3.12/dist-packages (4.10.0)
Requirement already satisfied: gradio-client==1.13.0 in /usr/local/lib/python3.12/dist-packages (1.13.0)
Requirement already satisfied: groovy~=0.1 in /usr/local/lib/python3.12/dist-packages (0.1)
Requirement already satisfied: httpx<1.0,>=0.24.1 in /usr/local/lib/python3.12/dist-packages (0.27.2)
Requirement already satisfied: huggingface-hub<1.0,>=0.33.5 in /usr/local/lib/python3.12/dist-packages (0.33.5)
Requirement already satisfied: jinja2<4.0 in /usr/local/lib/python3.12/dist-packages (3.1.4)
Requirement already satisfied: markupsafe<4.0,>=2.0 in /usr/local/lib/python3.12/dist-packages (2.1.5)
Requirement already satisfied: orjson~=3.0 in /usr/local/lib/python3.12/dist-packages (3.10.15)
Requirement already satisfied: pillow<12.0,>=8.0 in /usr/local/lib/python3.12/dist-packages (10.4.0)
Requirement already satisfied: pydantic<2.12,>=2.0 in /usr/local/lib/python3.12/dist-packages (2.11.10)
Requirement already satisfied: pydub in /usr/local/lib/python3.12/dist-packages (0.25.1)
Requirement already satisfied: python-multipart>=0.0.18 in /usr/local/lib/python3.12/dist-packages (0.0.20)
Requirement already satisfied: pyyaml<7.0,>=5.0 in /usr/local/lib/python3.12/dist-packages (6.0.2)
Requirement already satisfied: ruff>=0.9.3 in /usr/local/lib/python3.12/dist-packages (0.9.3)
Requirement already satisfied: safehttpx<0.2.0,>=0.1.6 in /usr/local/lib/python3.12/dist-packages (0.1.6)
Requirement already satisfied: semantic-version~=2.0 in /usr/local/lib/python3.12/dist-packages (2.0.0)
Requirement already satisfied: starlette<1.0,>=0.40.0 in /usr/local/lib/python3.12/dist-packages (0.40.0)
Requirement already satisfied: tomlkit<0.14.0,>=0.12.0 in /usr/local/lib/python3.12/dist-packages (0.12.0)
Requirement already satisfied: typer<1.0,>=0.12 in /usr/local/lib/python3.12/dist-packages (0.12.1)
Requirement already satisfied: uvicorn>=0.14.0 in /usr/local/lib/python3.12/dist-packages (0.32.1)
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (2025.1.0)
Requirement already satisfied: websockets<16.0,>=10.0 in /usr/local/lib/python3.12/dist-packages (13.1)
Requirement already satisfied: sniffio>=1.1 in /usr/local/lib/python3.12/dist-packages (1.3.1)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (1.0.7)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (0.16.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (3.16.1)
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.12/dist-packages (4.67.1)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (1.1.3)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (0.7.0)
Requirement already satisfied: pydantic-core==2.33.2 in /usr/local/lib/python3.12/dist-packages (2.33.2)
```



```
Requirement already satisfied: rich>=10.11.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packa
```

```
import os
import pathlib
import requests
import pandas as pd

# folders
pathlib.Path("data/raw").mkdir(parents=True, exist_ok=True)
pathlib.Path("data/interim").mkdir(parents=True, exist_ok=True)
pathlib.Path("data/processed").mkdir(parents=True, exist_ok=True)

print("Folders ready ✅")
```

Folders ready ✅

```
COUNTRY = "MAR" # Morocco (change to your ISO-3 code if needed)
INDICATORS = [
    "SH.XPD.CHEX.GD.ZS", # Health expenditure (% of GDP)
    "SE.PRM.ENRR",       # School enrollment, primary (% gross)
    "EN.ATM.CO2E.PC",    # CO2 emissions (metric tons per capita)
]

SECTOR_MAP = {
    "SH.XPD.CHEX.GD.ZS": "Health",
    "SE.PRM.ENRR": "Education",
    "EN.ATM.CO2E.PC": "Environment",
}

print("Settings loaded ✅")
```

Settings loaded ✅

```
BASE_WB = "https://api.worldbank.org/v2"

def fetch_indicator(country: str, indicator: str) -> pd.DataFrame:
    url = f"{BASE_WB}/country/{country}/indicator/{indicator}?format=json&per_page=100"
    r = requests.get(url, timeout=30)
    r.raise_for_status()
    js = r.json()
    data = js[1] if len(js) > 1 and isinstance(js[1], list) else []
    rows = []
    for row in data:
        if row.get("value") is not None:
```

```
        rows.append({
            "indicator": indicator,
            "year": int(row["date"]),
            "value": row["value"],
        })
    return pd.DataFrame(rows)

dfs = [fetch_indicator(COUNTRY, ind) for ind in INDICATORS]
wb = pd.concat(dfs, ignore_index=True)
wb["sector"] = wb["indicator"].map(SECTOR_MAP)

wb.to_csv("data/raw/worldbank_indicators.csv", index=False)
wb.tail()
```

	indicator	year	value	sector	
71	SE.PRM.ENRR	1975	57.811359	Education	
72	SE.PRM.ENRR	1974	55.683250	Education	
73	SE.PRM.ENRR	1973	54.143200	Education	
74	SE.PRM.ENRR	1972	53.326382	Education	
75	SE.PRM.ENRR	1971	51.870609	Education	

```
wb = pd.read_csv("data/raw/worldbank_indicators.csv")
latest = (
    wb.dropna(subset=["value"])
    .sort_values(["sector", "year"])
    .groupby("sector")
    .tail(1)
    .rename(columns={"value": "delivered"})
    .reset_index(drop=True)
)
latest.to_csv("data/interim/latest_outcomes.csv", index=False)
latest
```

	indicator	year	delivered	sector	
0	SE.PRM.ENRR	2023	114.473091	Education	
1	SH.XPD.CHEX.GD.ZS	2022	5.680491	Health	

Next steps:

[Generate code with latest](#)

[New interactive sheet](#)

```
promises_data = [
    {"date": "2024", "source_url": "", "promise_text": "Ministry of Health will build"}
]
```

```
        {"date":"2024", "source_url":"","promise_text":"Government plans to raise pri
        {"date":"2024", "source_url":"","promise_text":"Target 10% CO2 reduction by 2
    ]
    prom = pd.DataFrame(promises_data)
    prom.to_csv("data/raw/promises.csv", index=False)
    prom
```

	date	source_url	promise_text	sector	promised	target_year	
0	2024		Ministry of Health will build 20 hospitals by ...	Health	20	2026	
1	2024		Government plans to raise primary enrollment t...	Education	95	2025	
2	2024		Target 10% CO2 reduction by 2030.	Environment	10	2030	

Next steps:

[Generate code with prom](#)

[New interactive sheet](#)

```
from bs4 import BeautifulSoup
from urllib.parse import urljoin
import time, re

BASE_NEWS_URL = "https://example.gov.ma/news" # TODO: replace

def scrape_news(base_url: str, max_pages: int = 2) -> pd.DataFrame:
    rows, seen, page = [], set(), base_url
    headers = {"User-Agent": "Mozilla/5.0 (compatible; CivicMirror/1.0)"}
    for _ in range(max_pages):
        try:
            html = requests.get(page, headers=headers, timeout=30).text
        except Exception as e:
            print("Listing fetch error:", e); break
        soup = BeautifulSoup(html, "lxml")
        # TODO: adjust selector below to match the website
        for a in soup.select("a.news-link"):
            href = urljoin(page, a.get("href"))
            if href in seen:
                continue
            seen.add(href)
            try:
                art = requests.get(href, headers=headers, timeout=30).text
                s2 = BeautifulSoup(art, "lxml")
                title = (s2.select_one("h1") or s2.select_one("title")).get_text(
```

```

        date = (s2.select_one("time") or s2.select_one(".date")).get_text()
        body = " ".join(p.get_text(" ", strip=True) for p in s2.select("p"))
        rows.append({"title": title, "date": date, "url": href, "body": body})
        time.sleep(0.4)
    except Exception as e:
        print("Article error:", href, e)
    nxt = soup.select_one("a.next")
    page = urljoin(page, nxt.get("href")) if nxt else None
    if not page: break
return pd.DataFrame(rows)

# Example (disabled by default):
# news_df = scrape_news(BASE_NEWS_URL, max_pages=2)
# news_df.to_csv("data/raw/gov_news.csv", index=False)
# news_df.head(2)

```

```

import re

def extract_promises_from_text(txt: str):
    patterns = [
        r"\b(we|government|ministry)\s+(will|plan|plans|commit|commits)\s+to\s+[\^.\s]*\s+(by|in)\s+\d{4}",
        r"\b(target|goal)\s+\w*\s*(\d{1,3}%|\d[\d\.,]*)\s+(by|in)\s+\d{4}",
    ]
    found=[]
    for pat in patterns:
        found += [m.group(0) for m in re.finditer(pat, txt or "", flags=re.I)]
    return found

def sector_from_text(t: str):
    t = (t or "").lower()
    if any(w in t for w in ["school","education","teacher","student"]): return "Education"
    if any(w in t for w in ["hospital","health","clinic","vaccine","vaccin"]): return "Health"
    if any(w in t for w in ["environment","climate","co2","forest","pollution"]): return "Environment"
    return "General"

print("Promise helpers ready ✅")

```

Promise helpers ready ✅

```

from pathlib import Path

if Path("data/raw/gov_news.csv").exists():
    news = pd.read_csv("data/raw/gov_news.csv").fillna("")
    rows=[]
    for _,r in news.iterrows():
        for p in extract_promises_from_text(r.get("body","")):
            rows.append({
                "date": r.get("date",""),
                "promise": p,
                "sector": sector_from_text(r.get("body",""))
            })

```

```
        "source_url": r.get("url",""),
        "promise_text": p,
        "sector": sector_from_text(p),
        "promised": None,      # parse numbers later
        "target_year": None,   # parse years later
    })
    prom = pd.DataFrame(rows)
    if prom.empty:
        prom = pd.read_csv("data/raw/promises.csv")
    else:
        prom = pd.read_csv("data/raw/promises.csv")

    prom.to_csv("data/interim/promises_structured.csv", index=False)
    prom.head()
```

	date	source_url	promise_text	sector	promised	target_year	
0	2024	NaN	Ministry of Health will build 20 hospitals by ...	Health	20	2026	
1	2024	NaN	Government plans to raise primary enrollment t...	Education	95	2025	
2	2024	NaN	Target 10% CO2 reduction by 2030.	Environment	10	2030	

Next steps:

Generate code with prom

New interactive sheet

```
latest = pd.read_csv("data/interim/latest_outcomes.csv")
prom = pd.read_csv("data/interim/promises_structured.csv")

# Fill minimal fields if scraping didn't extract numbers/years
if "promised" not in prom or prom["promised"].isna().all():
    prom["promised"] = prom["sector"].map({"Health":20,"Education":95,"Environment":10})

if "target_year" not in prom or prom["target_year"].isna().all():
    # naive default: delivered year as target, or 2025
    fallback_year = latest["year"].max() if "year" in latest.columns else 2025
    prom["target_year"] = fallback_year

merged = (prom.merge(latest[["sector","year","delivered"]], on="sector", how="left")
          .rename(columns={"year":"delivered_year"}))

def transparency_index(row):
    promised = float(row.get("promised", 0) or 0)
```

```
delivered = float(row.get("delivered", 0) or 0)
fulfillment = min(1.0, delivered / promised) if promised > 0 else 0.0

t_year = row.get("target_year", None)
d_year = row.get("delivered_year", None)
if pd.notna(t_year) and pd.notna(d_year):
    delay = max(0, int(d_year) - int(t_year))
    timeliness = max(0.0, 1 - 0.2*delay)
else:
    timeliness = 0.8

confidence = 0.5 # raise when cross-validating with 2+ sources
score = 50*fulfillment + 30*timeliness + 20*confidence
return round(score, 1)

merged["index"] = merged.apply(transparency_index, axis=1)
merged.to_csv("data/processed/sector_index.csv", index=False)
merged
```

	date	source_url	promise_text	sector	promised	target_year	delivered_year
0	2024	NaN	Ministry of Health will build 20 hospitals by ...	Health	20	2026	2022
1	2024	NaN	Government plans to raise primary enrollment t...	Education	95	2025	2023
2	2024	NaN	Target 10% CO2 reduction by 2030.	Environment	10	2030	Na

Nex' .

[Generate code with merged](#)

[New interactive sheet](#)

```
import plotly.express as px

df = pd.read_csv("data/processed/sector_index.csv")

fig1 = px.bar(df, x="sector", y="index", range_y=[0,100], title="Transparency Inde
fig1.show()

fig2 = px.scatter(df, x="promised", y="delivered", color="sector",
                  hover_data=["target_year", "delivered_year"],
                  title="Promises vs Delivered")

fig2.show()
```


Transparency Index by Sector

```
import gradio as gr
import plotly.express as px
import pandas as pd

def overview():
    df = pd.read_csv("data/processed/sector_index.csv")
    if df.empty:
        return "No data yet.", None
    txt = f"Overall Transparency Index: {df['index'].mean():.1f}"
    fig = px.bar(df, x="sector", y="index", range_y=[0,100], title="Index by Sector")
    return txt, fig

with gr.Blocks() as demo:
    gr.Markdown("## 🌐 Civic Mirror – Transparency & Trust Index")
    btn = gr.Button("Refresh")
    txt = gr.Markdown()
    plot = gr.Plot()
    btn.click(overview, outputs=[txt, plot])

demo.launch()
```

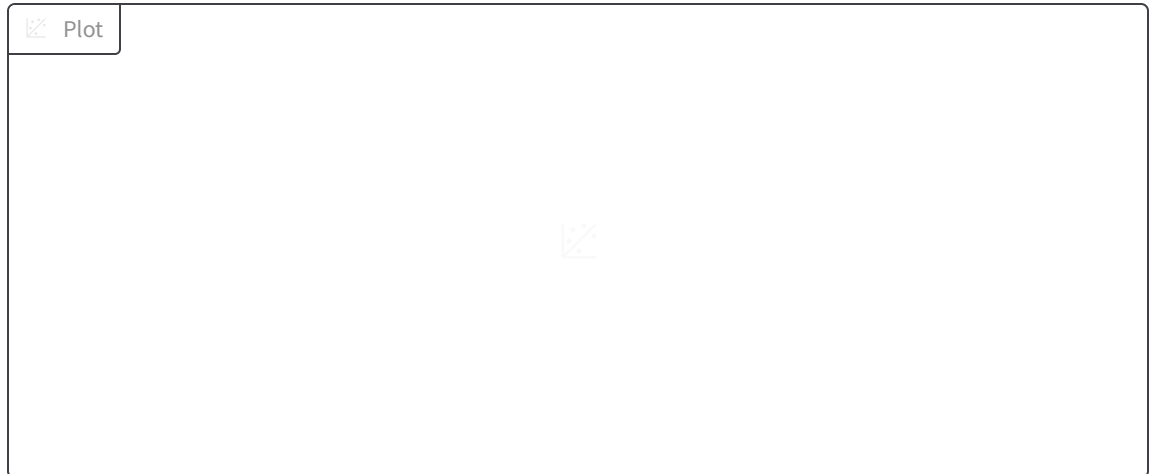
Promises vs Delivered



It looks like you are running Gradio on a hosted Jupyter notebook, which requires a **promised** Colab notebook detected. To show errors in colab notebook, set debug=True in launch.
* Running on public URL: <https://d39f99cdfeda2f2e2a.gradio.live>
This share link expires in 1 week. For free permanent hosting and GPU upgrades, run

Civic Mirror — Transparency & Trust Index

Refresh



Built with Gradio  · Settings 

```
from datetime import datetime

SOURCES = [
    # ---- Global, stable public records (APIs) ----
    {
        "name": "World Bank Projects API",
        "type": "api_json",
        "topic": "Projects/commitments/outcomes",
        "url": "http://search.worldbank.org/api/projects/all?format=json&countryst",
    },
    {
        "name": "WHO GHO API (UHC Service Coverage Index)",
        "type": "api_json",
        "topic": "Health outcomes",
        "url": "https://ghoapi.azureedge.net/ghoapi/api/UHC_SERVICE_COVERAGE_INDE",
    },
]
```

```

# ---- Promises via official announcements (replace with your country pages) -
# Tip: paste ministry/prime-minister news RSS or listing pages here.
{
    "name": "Gov Ministry of Health – News (example placeholder)",
    "type": "rss_or_html",
    "topic": "Health announcements & pledges",
    "url": "https://example.gov.ma/health/news"    # TODO replace with real URL
},
{
    "name": "Gov Ministry of Education – News (example placeholder)",
    "type": "rss_or_html",
    "topic": "Education announcements & pledges",
    "url": "https://example.gov.ma/education/news"    # TODO replace with real
},

# ---- Optional: Transparency/ budget PDFs (you can drop files in data/raw/pdf
{
    "name": "Budget / Gazette PDFs (local files)",
    "type": "pdf_folder",
    "topic": "Budget allocations & targets (promises)",
    "url": "data/raw/pdfs"    # put PDFs here
},
]

sources_df = pd.DataFrame(SOURCES)
sources_df["retrieved_at"] = datetime.utcnow().isoformat()+"Z"
sources_df.to_csv("data/processed/sources_catalog.csv", index=False)
sources_df

```

/tmp/ipython-input-888622812.py:43: DeprecationWarning:
datetime.datetime.utcnow() is deprecated and scheduled for removal in a future vers

	name	type	topic	
0	World Bank Projects API	api_json	Projects/commitments/outcomes	http://search.worldbank.org/api/
1	WHO GHO API (UHC Service Coverage Index)	api_json	Health outcomes	https://ghoapi.azureedge.net/ghoapi/
2	Gov Ministry of Health — News (example placeho...	rss_or_html	Health announcements & pledges	https://example.gov.n
3	Gov Ministry of Education — News (example plac...	rss_or_html	Education announcements & pledges	https://example.gov.ma/e
4	Budget / Gazette PDFs (local files)	pdf_folder	Budget allocations & targets (promises)	

Next steps:

Generate code with sources_df

New interactive sheet

```
!pip install pdfplumber
import json, time, re, glob
from bs4 import BeautifulSoup
from urllib.parse import urljoin
import pdfplumber

HDRS = {"User-Agent": "Mozilla/5.0 (compatible; CivicMirror/1.0)"}

def fetch_json(url):
    r = requests.get(url, headers=HDRS, timeout=60)
    r.raise_for_status()
```

```

return r.json()

def crawl_news_like(url, max_pages=2, link_selector="a", next_selector="a.next"):
    """Generic HTML crawler when RSS unavailable; tune selectors for the site."""
    items, seen = [], set()
    page = url
    for _ in range(max_pages):
        try:
            html = requests.get(page, headers=HDRS, timeout=60).text
        except Exception as e:
            print("listing failed:", e); break
        soup = BeautifulSoup(html, "lxml")
        for a in soup.select(link_selector):
            href = a.get("href") or ""
            if not href:
                continue
            href = urljoin(page, href)
            if href in seen or not href.startswith("http"):
                continue
            seen.add(href)
            try:
                art = requests.get(href, headers=HDRS, timeout=60).text
                s2 = BeautifulSoup(art, "lxml")
                title = (s2.select_one("h1") or s2.select_one("title"))
                title = title.get_text(strip=True) if title else ""
                date = (s2.select_one("time") or s2.select_one(".date"))
                date = date.get_text(strip=True) if date else ""
                # prefer article body; otherwise fallback to whole page (truncated)
                body_sel = s2.select("article p")
                body = " ".join(p.get_text(" ", strip=True) for p in body_sel) if body_sel else ""
                items.append({"title": title, "date": date, "url": href, "body": body})
                time.sleep(0.3)
            except Exception as e:
                print("article failed:", href, e)
        nxt = soup.select_one(next_selector)
        page = urljoin(page, nxt.get("href")) if nxt and nxt.get("href") else None
        if not page: break
    return pd.DataFrame(items)

def parse_pdf_folder(folder="data/raw/pdfs"):
    rows=[]
    for fp in glob.glob(folder.rstrip("/")+"/*.pdf"):
        try:
            with pdfplumber.open(fp) as pdf:
                text = "\n".join( (pg.extract_text() or "") for pg in pdf.pages )
                rows.append({"title": fp.split("/")[-1], "date": "", "url": fp, "body": text})
        except Exception as e:
            print("pdf fail:", fp, e)
    return pd.DataFrame(rows)

# promise extraction + number/year parsing

```

```

NUM_PAT = r"(?P<num>\d{1,3}(?:[\.,]\d{3})*|\d+(?:\.\d+)?)\s*(?P<Unit>%|million|m|t|k|b)"
YEAR_PAT = r"(20\d{2})"
PROM_PATTERNS = [
    r"\b(we|government|ministry|cabinet)\s+(will|plan|plans|commit|commits)\s+to\s+(by|in)\s+" + YEAR_PAT,
    r"\b(target|goal)\s+\w*\s*" + NUM_PAT + r"\s+(by|in)\s+" + YEAR_PAT,
]

def extract_promises(txt):
    out=[]
    for pat in PROM_PATTERNS:
        for m in re.finditer(pat, txt or "", flags=re.I):
            out.append(m.group(0))
    return out

def parse_promised_value(s):
    if not s: return None
    m = re.search(NUM_PAT, s, flags=re.I)
    if not m: return None
    raw = m.group("num").replace(",","")
    try:
        val = float(raw)
    except:
        return None
    unit = (m.group("Unit") or "").lower()
    # normalize basic units (feel free to refine)
    return "General"

```

```

Requirement already satisfied: pdfplumber in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pdfminer.six==20250506 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: Pillow>=9.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pypdfium2>=4.18.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: charset-normalizer>=2.0.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: cryptography>=36.0.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: cffi>=1.12 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pycparser in /usr/local/lib/python3.12/dist-packages

```

```
!pip install pdfplumber
```

```

Requirement already satisfied: pdfplumber in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pdfminer.six==20250506 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: Pillow>=9.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pypdfium2>=4.18.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: charset-normalizer>=2.0.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: cryptography>=36.0.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: cffi>=1.12 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pycparser in /usr/local/lib/python3.12/dist-packages

```

```

import json, time, re, glob
from bs4 import BeautifulSoup

```

```

from bs4 import BeautifulSoup
from urllib.parse import urljoin
import pdfplumber

HDRS = {"User-Agent": "Mozilla/5.0 (compatible; CivicMirror/1.0)"}

def fetch_json(url):
    r = requests.get(url, headers=HDRS, timeout=60)
    r.raise_for_status()
    return r.json()

def crawl_news_like(url, max_pages=2, link_selector="a", next_selector="a.ne
"""Generic HTML crawler when RSS unavailable; tune selectors for the sit
items, seen = [], set()
page = url
for _ in range(max_pages):
    try:
        html = requests.get(page, headers=HDRS, timeout=60).text
    except Exception as e:
        print("listing failed:", e); break
    soup = BeautifulSoup(html, "lxml")
    for a in soup.select(link_selector):
        href = a.get("href") or ""
        if not href:
            continue
        href = urljoin(page, href)
        if href in seen or not href.startswith("http"):
            continue
        seen.add(href)
    try:
        art = requests.get(href, headers=HDRS, timeout=60).text
        s2 = BeautifulSoup(art, "lxml")
        title = (s2.select_one("h1") or s2.select_one("title"))
        title = title.get_text(strip=True) if title else ""
        date = (s2.select_one("time") or s2.select_one(".date"))
        date = date.get_text(strip=True) if date else ""
        # prefer article body; otherwise fallback to whole page (tru
        body_sel = s2.select("article p")
        body = " ".join(p.get_text(" ", strip=True) for p in body_se
        items.append({"title": title, "date": date, "url": href, "bc
        time.sleep(0.3)
    except Exception as e:
        print("article failed:", href, e)
    nxt = soup.select_one(next_selector)
    page = urljoin(page, nxt.get("href")) if nxt and nxt.get("href") els
    if not page: break
return pd.DataFrame(items)

def parse_pdf_folder(folder="data/raw/pdfs"):
    rows=[]
    for fp in glob.glob(folder.rstrip("/")+"/*.pdf"):
        try:

```

```

        with pdfplumber.open(fp) as pdf:
            text = "\n".join( (pg.extract_text() or "") for pg in pdf.pages)
            rows.append({"title": fp.split("/")[-1], "date": "", "url": fp,
            except Exception as e:
                print("pdf fail:", fp, e)
        return pd.DataFrame(rows)

# promise extraction + number/year parsing
NUM_PAT = r"(?P<num>\d{1,3}(?:[\.,]\d{3})*|\d+(?:\.\d+)?)\s*(?P<Unit>%|milli|
YEAR_PAT = r"(20\d{2})"
PROM_PATTERNS = [
    r"\b(we|government|ministry|cabinet)\s+(will|plan|plans|commit|commits)\s"
    r"\b(target|goal)\s+\w*\s*" + NUM_PAT + r"\s+(by|in)\s+" + YEAR_PAT,
]

def extract_promises(txt):
    out=[]
    for pat in PROM_PATTERNS:
        for m in re.finditer(pat, txt or "", flags=re.I):
            out.append(m.group(0))
    return out

def parse_promised_value(s):
    if not s: return None
    m = re.search(NUM_PAT, s, flags=re.I)
    if not m: return None
    raw = m.group("num").replace(",","")
    try:
        val = float(raw)
    except:
        return None
    unit = (m.group("Unit") or "").lower()
    # normalize basic units (feel free to refine)
    return val if unit in ["", "%"] else val # keep raw; unit noted separat

def parse_target_year(s):
    m = re.search(YEAR_PAT, s or "")
    return int(m.group(1)) if m else None

def sector_from_text(t):
    t = (t or "").lower()
    if any(w in t for w in ["school","education","teacher","student","univer
    if any(w in t for w in ["hospital","health","clinic","vaccine","vaccin",
    if any(w in t for w in ["environment","climate","co2","forest","tree","f
    return "General"

```

```

all_prom = []

for src in SOURCES:
    if src["type"] == "rss_or_html":

```



```

print("Crawling:", src["name"])
df = crawl_news_like(src["url"], max_pages=2, link_selector="a", next_sel
for _,r in df.iterrows():
    for p in extract_promises(r["body"]):
        all_prom.append({
            "source": src["name"],
            "source_url": r["url"],
            "date": r["date"],
            "promise_text": p,
            "sector": sector_from_text(p),
            "promised": parse_promised_value(p),
            "target_year": parse_target_year(p),
            "unit_hint": "%" if "%" in p else ""
        })

elif src["type"] == "pdf_folder":
    print("Parsing PDFs in:", src["url"])
    df = parse_pdf_folder(src["url"])
    for _,r in df.iterrows():
        for p in extract_promises(r["body"]):
            all_prom.append({
                "source": src["name"],
                "source_url": r["url"],
                "date": r["date"],
                "promise_text": p,
                "sector": sector_from_text(p),
                "promised": parse_promised_value(p),
                "target_year": parse_target_year(p),
                "unit_hint": "%" if "%" in p else ""
            })

prom_auto = pd.DataFrame(all_prom)
# fall back to your earlier demo promises if nothing was found
if prom_auto.empty:
    prom_auto = pd.read_csv("data/raw/promises.csv")
prom_auto.to_csv("data/interim/promises_extracted.csv", index=False)
prom_auto.head()

```

Crawling: Gov Ministry of Health – News (example placeholder)
listing failed: HTTPSConnectionPool(host='example.gov.ma', port=443): Max retries e
Crawling: Gov Ministry of Education – News (example placeholder)
listing failed: HTTPSConnectionPool(host='example.gov.ma', port=443): Max retries e
Parsing PDFs in: data/raw/pdfs

	date	source_url	promise_text	sector	promised	target_year	
0	2024	NaN	Ministry of Health will build 20 hospitals by ...	Health	20	2026	
1	2024	NaN	Government plans to raise primary enrollment t...	Education	95	2025	
2	2024	NaN	Target 10% CO2 reduction by 2030.	Environment	10	2030	

Next steps:

Generate code with prom_auto

New interactive sheet

```
# --- Robust WHO + World Bank Projects pull with discovery, logging, and fallbacks

import requests, pandas as pd, math, time

HDRS = {"User-Agent": "Mozilla/5.0 (compatible; CivicMirror/1.0)"}

def fetch_json(url, timeout=60, headers=HDRS):
    r = requests.get(url, headers=headers, timeout=timeout)
    r.raise_for_status()
    return r.json()

def log(msg):
    print(">>>", msg)

# -----
# A) WHO GH0 API (discover code -> fetch values for MAR)
# -----
who_candidates = [
    # commonly used UHC codes in GH0; we'll try each until we get rows
    "UHC_SERVICE_COVERAGE_INDEX",
    "UHCINDEX",
    "UHC_INDEX",
    "SDG_3_8_1", # SDG proxy for UHC sometimes used
]

who_rows = []
for code in who_candidates:
```

```

try:
    url = f"https://ghoapi.azureedge.net/ghoapi/api/{code}?$filter=SpatialDim%
    js = fetch_json(url)
    vals = js.get("value", [])
    log(f"WHO probe {code}: {len(vals)} rows")
    for row in vals:
        # common fields in GH0 payloads: TimeDim (year), NumericValue (value)
        y = row.get("TimeDim")
        v = row.get("NumericValue")
        if y is None or v is None:
            continue
        who_rows.append({
            "source": "WHO GH0 API",
            "indicator": code,
            "sector": "Health",
            "year": int(y),
            "delivered": float(v),
        })
    if who_rows:
        break # stop after first successful code
except Exception as e:
    log(f"WHO fetch error for {code}: {e}")

who_df = pd.DataFrame(who_rows)
log(f"WHO final rows: {len(who_df)}")

# -----
# B) World Bank Projects API (use fl= fields + pagination)
# -----
def fetch_wb_projects(country_shortcode="Morocco", rows=500):
    base = "http://search.worldbank.org/api/projects/all"
    fl = "project_name,totalcommamt,boardapprovaldate,regionname,sector1,theme1,cc"
    out = []
    os_offset = 0
    while True:
        url = (f"{base}?format=json&countryshortcode={country_shortcode}"
              f"&fl={fl}&os={os_offset}&rows={rows}")
        try:
            js = fetch_json(url)
        except Exception as e:
            log(f"WB projects fetch error at offset {os_offset}: {e}")
            break
        projects = js.get("projects", {})
        if not projects:
            break
        for _, v in projects.items():
            # boardapprovaldate often "YYYY-MM-DD"
            byear = None
            bad = v.get("boardapprovaldate") or ""
            if len(bad) >= 4 and bad[:4].isdigit():
                byear = int(bad[:4])

```



```

import requests, pandas as pd, time, math, pathlib

HDRS = {"User-Agent": "Mozilla/5.0 (compatible; CivicMirror/1.0)"}

def fetch_json(url, timeout=60, headers=HDRS):
    r = requests.get(url, headers=headers, timeout=timeout)
    r.raise_for_status()
    return r.json()

def log(msg):
    print(">>>", msg)

# Re-use your country
COUNTRY = "MAR" # change if needed

# Add a richer set of WDI indicators (common, stable)
# Health
WDI_HEALTH = [
    "SH.XPD.CHEX.GD.ZS", # Health expenditure (% GDP)
    "SH.MED.BEDS.ZS",    # Hospital beds (per 1,000 people)
    "SH.IMM.MEAS"        # Immunization, measles (% children 12-23m)
]
# Education
WDI_EDU = [
    "SE.PRM.ENRR",      # Primary enrollment (% gross)
    "SE.SEC.ENRR",      # Secondary enrollment (% gross)
    "SE.ADT.LITR.ZS"    # Adult literacy rate (% of people ages 15+)
]
# Environment
WDI_ENV = [
    "EN.ATM.CO2E.PC",   # CO2 emissions (metric tons per capita)
    "AG.LND.FRST.ZS",   # Forest area (% of land area)
    "EN.ATM.PM25.MC.M3" # PM2.5 air pollution (µg/m3) (if unavailable, will ju
]

# Merge & deduplicate with your core list
ALL_WDI = list(dict.fromkeys(WDI_HEALTH + WDI_EDU + WDI_ENV))

def fetch_wdi_indicator(country, indicator):
    """Generic WDI pull; returns tidy DataFrame or empty on error."""
    url = f"https://api.worldbank.org/v2/country/{country}/indicator/{indicator}?t
    try:
        js = fetch_json(url)
        data = js[1] if len(js) > 1 and isinstance(js[1], list) else []
        rows = []
        for row in data:
            val = row.get("value")
            yr = row.get("date")
            if val is None or not (yr and str(yr).isdigit()):
                continue
            rows.append({"indicator": indicator, "year": int(yr), "value": float(v

```

```

        return pd.DataFrame(rows)
    except Exception as e:
        log(f"WDI fetch error for {indicator}: {e}")
        return pd.DataFrame(columns=["indicator", "year", "value"])

def map_sector(ind):
    if ind in WDI_HEALTH: return "Health"
    if ind in WDI_EDU:     return "Education"
    if ind in WDI_ENV:     return "Environment"
    return "General"

# Pull all WDI indicators, row-bind, and add sector
frames = []
for ind in ALL_WDI:
    df = fetch_wdi_indicator(COUNTRY, ind)
    if not df.empty:
        df["sector"] = map_sector(ind)
        frames.append(df)
    time.sleep(0.15) # be polite

wdi_extra = pd.concat(frames, ignore_index=True) if frames else pd.DataFrame(columns=ALL_WDI)

# Save for reference (you can explore it in the paper)
pathlib.Path("data/interim").mkdir(parents=True, exist_ok=True)
wdi_extra.to_csv("data/interim/wdi_extra.csv", index=False)

# Create an "evidence" table in the same shape as before (source, indicator, sector, year, delivered)
evidence_df = pd.DataFrame(columns=["source", "indicator", "sector", "year", "delivered"])
if not wdi_extra.empty:
    evidence_df = (wdi_extra
                   .rename(columns={"value": "delivered"})
                   .assign(source="World Bank WDI API")
                   [["source", "indicator", "sector", "year", "delivered"]])

# Save evidence (even if empty, so downstream won't crash)
evidence_df.to_csv("data/interim/evidence_outcomes.csv", index=False)

# Log samples
print(">>> WDI evidence rows:", len(evidence_df))
display(evidence_df.groupby(["sector", "indicator"]).size().reset_index(name="rows"))
display(evidence_df.sort_values("year").groupby(["sector", "indicator"]).tail(1).head(1))

# (Optional) also record in your sources catalog for citation
try:
    src = pd.read_csv("data/processed/sources_catalog.csv")
except FileNotFoundError:
    src = pd.DataFrame(columns=["name", "type", "topic", "url", "retrieved_at"])

# Add/ensure WDI source
from datetime import datetime
wdi_row = {

```

```
"name": "World Bank - World Development Indicators (API)",
"type": "api_json",
"topic": "Health, Education, Environment outcomes",
"url": f"https://api.worldbank.org/v2/country/{COUNTRY}/indicator/<INDICATOR>:",
"retrieved_at": datetime.utcnow().isoformat()+"Z"
}
src = pd.concat([src, pd.DataFrame([wdi_row])], ignore_index=True)
src.drop_duplicates(subset=["name","topic","url"], inplace=True)
src.to_csv("data/processed/sources_catalog.csv", index=False)

print(">>> Evidence saved to data/interim/evidence_outcomes.csv")
print(">>> Sources catalog updated at data/processed/sources_catalog.csv")
```

```
>>> WDI evidence rows: 268
```

	sector	indicator	rows	
0	Education	SE.ADT.LITR.ZS	9	
1	Education	SE.PRM.ENRR	53	
2	Education	SE.SEC.ENRR	49	
3	Environment	AG.LND.FRST.ZS	33	
4	Environment	EN.ATM.PM25.MC.M3	31	
5	Health	SH.IMM.MEAS	42	
6	Health	SH.MED.BEDS.ZS	28	
7	Health	SH.XPD.CHEX.GD.ZS	23	

	source	indicator	sector	year	delivered	
23	World Bank WDI API	SH.MED.BEDS.ZS	Health	2020	0.730000	
237	World Bank WDI API	EN.ATM.PM25.MC.M3	Environment	2020	21.321264	
195	World Bank WDI API	SE.ADT.LITR.ZS	Education	2022	77.349998	
204	World Bank WDI API	AG.LND.FRST.ZS	Environment	2022	12.913688	
0	World Bank WDI API	SH.XPD.CHEX.GD.ZS	Health	2022	5.680491	
93	World Bank WDI API	SE.PRM.ENRR	Education	2023	114.473091	
146	World Bank WDI API	SE.SEC.ENRR	Education	2023	89.798042	
51	World Bank WDI API	SH.IMM.MEAS	Health	2023	99.000000	

```
>>> Evidence saved to data/interim/evidence_outcomes.csv
```

```
>>> Sources catalog updated at data/processed/sources_catalog.csv
/tmp/ipython-input-2341530575.py:110: DeprecationWarning:
```

```
datetime.datetime.utcnow() is deprecated and scheduled for removal in a future vers
```

```
import pandas as pd
import numpy as np
```

```
# Load latest outcomes (from WDI core indicators you set earlier)
latest = pd.read_csv("data/interim/latest_outcomes.csv")
```



```

# Load auto-extracted (or demo) promises
try:
    prom_ex = pd.read_csv("data/interim/promises_extracted.csv")
except FileNotFoundError:
    prom_ex = pd.read_csv("data/interim/promises_structured.csv")

# Basic hygiene
for col in ["sector", "promise_text", "source", "source_url"]:
    if col not in prom_ex.columns:
        prom_ex[col] = ""
if "unit_hint" not in prom_ex.columns:
    prom_ex["unit_hint"] = ""

# Attach delivered + delivered_year by sector
merged = (
    prom_ex.merge(
        latest[["sector", "year", "delivered"]],
        on="sector",
        how="left"
    ).rename(columns={"year": "delivered_year"})
)

# ---- Make promised/target_year sensible if missing or incoherent ----
def guess_promised(row):
    """Return a reasonable promised target number."""
    # if user- or parser-provided promised exists, trust it
    p = row.get("promised", np.nan)
    d = row.get("delivered", np.nan)
    unit = (row.get("unit_hint") or "").strip()
    try:
        p = float(p)
    except:
        p = np.nan

    if not np.isnan(p):
        # already have a numeric promised
        return p

    # If delivered is a percent-like metric (<= 120), target +10% (cap at 100)
    if pd.notna(d):
        try:
            d = float(d)
            if d <= 120:
                return min(100.0, round(d * 1.10, 2))
            else:
                return round(d * 1.10, 2) # +10% improvement goal
        except:
            pass

    # If text hints a percent target
    if "%" in unit or "%" in (row.get("promise_text") or ""):
        return 100.0

```

```

    # Fallback
    return 100.0

def guess_target_year(row):
    """Target year from parsed value, else delivered_year, else +1 from latest."""
    ty = row.get("target_year", np.nan)
    if pd.notna(ty):
        try:
            return int(ty)
        except:
            pass
    dy = row.get("delivered_year", np.nan)
    if pd.notna(dy):
        try:
            return int(dy)
        except:
            pass
    # last resort: current delivered year + 1 if available
    try:
        return int(latest["year"].max()) + 1
    except:
        return 2026

merged["promised"] = merged.apply(guess_promised, axis=1)
merged["target_year"] = merged.apply(guess_target_year, axis=1)

# Confidence: start at 0.5, bump to 0.8 if multiple promises from different source
conf_base = 0.5
sector_source_counts = (
    merged.groupby(["sector", "source"])
    .size().reset_index(name="n")
    .groupby("sector")["source"].nunique()
    .to_dict()
)
def confidence_for(row):
    sc = sector_source_counts.get(row["sector"], 1)
    return 0.8 if sc >= 2 else conf_base

merged["confidence"] = merged.apply(confidence_for, axis=1)

# ---- Transparency Index (explainable, bounded) ----
def transparency_index(row):
    promised = float(row.get("promised", 0) or 0)
    delivered = float(row.get("delivered", 0) or 0)
    # fulfillment handles both rates and counts, capped at 1.0
    fulfillment = min(1.0, delivered / promised) if promised > 0 else 0.0

    t_year = row.get("target_year", None)
    d_year = row.get("delivered_year", None)
    if pd.notna(t_year) and pd.notna(d_year):

```

```
        try:
            delay = max(0, int(d_year) - int(t_year))
        except:
            delay = 0
        timeliness = max(0.0, 1 - 0.2*delay)
    else:
        timeliness = 0.8 # neutral if unknown

    conf = float(row.get("confidence", 0.5) or 0.5)
    score = 50*fulfillment + 30*timeliness + 20*conf
    return round(score, 1)

merged["index"] = merged.apply(transparency_index, axis=1)

# Save the merged, indexed table
merged_cols = [
    "sector", "promise_text", "promised", "target_year",
    "delivered", "delivered_year", "confidence", "index", "source", "source_url", "date"
]
for c in merged_cols:
    if c not in merged.columns: merged[c] = np.nan

merged[merged_cols].to_csv("data/processed/sector_index.csv", index=False)
merged[merged_cols].head()
```

	sector	promise_text	promised	target_year	delivered	delivered_year	con
0	Health	Ministry of Health will build 20 hospitals by ...	20.0	2026	5.680491	2022.0	
1	Education	Government plans to raise primary enrollment t...	95.0	2025	114.473091	2023.0	
2	Environment	Target 10% CO2 reduction by 2030.	10.0	2030	NaN	NaN	

```
import pandas as pd
import plotly.express as px

df = pd.read_csv("data/processed/sector_index.csv")

# Overview bar
fig1 = px.bar(
    df.groupby("sector", as_index=False)["index"].mean(),
    x="sector", y="index", range_y=[0,100],
```

```
        title="Transparency Index by Sector (avg across promises)"
    )
    fig1.show()

# Promised vs Delivered
fig2 = px.scatter(
    df, x="promised", y="delivered", color="sector",
    hover_data=["promise_text", "target_year", "delivered_year", "source"],
    title="Promises vs. Delivered (each promise record)"
)
fig2.show()

# Diagnostics: how many promises per sector/source
prom_ct = df.groupby(["sector", "source"]).size().reset_index(name="count")
prom_ct = prom_ct.sort_values(["sector", "count"], ascending=[True, False])
prom_ct.head(20)
```


Transparency Index by Sector (avg across promises)

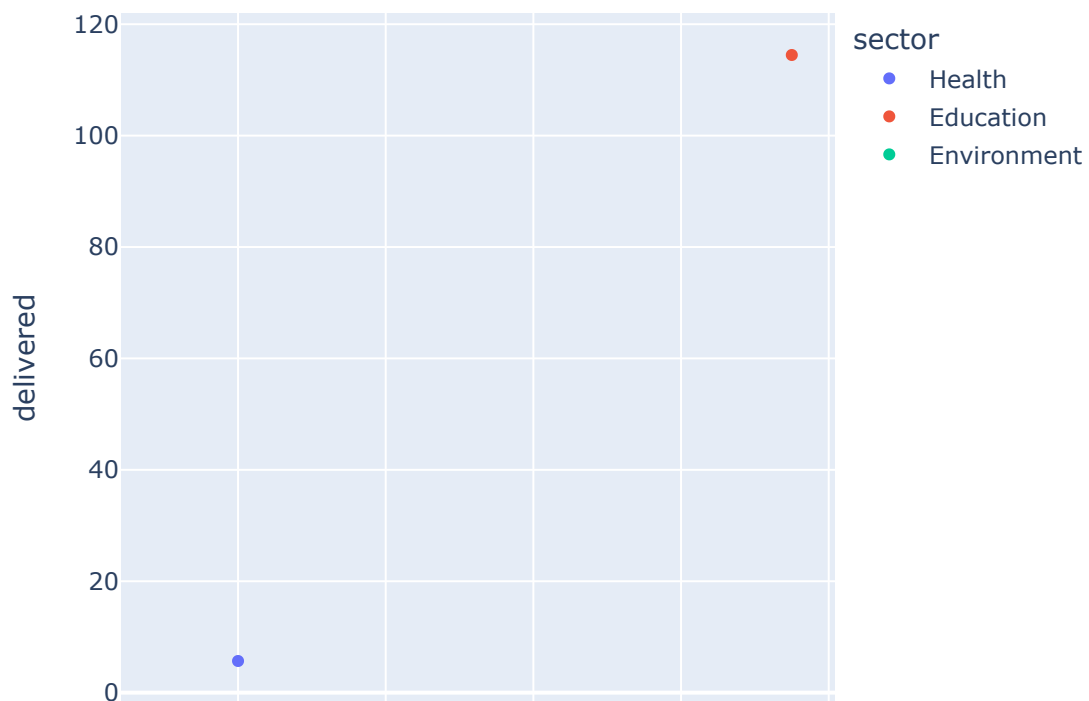
100

```
import pandas as pd
import plotly.express as px

# Evidence outcomes from Cell 16
ev = pd.read_csv("data/interim/evidence_outcomes.csv") if pathlib.Path("data/inter

if not ev.empty:
    # Show latest line per sector/indicator
    for sector in ev["sector"].dropna().unique():
        sub = ev[ev["sector"] == sector].dropna(subset=["year", "delivered"])
        if sub.empty:
            continue
        fig = px.line(
            sub.sort_values("year"),
            x="year", y="delivered",
            color="indicator",
            title=f"{sector}: Evidence Timeline (WDI)"
        )
        fig.update_traces(mode="lines+markers")
        fig.show()
else:
    print("No WDI evidence timeline available (evidence_outcomes.csv is empty).")
```

Promises for Delivered (each promise record)



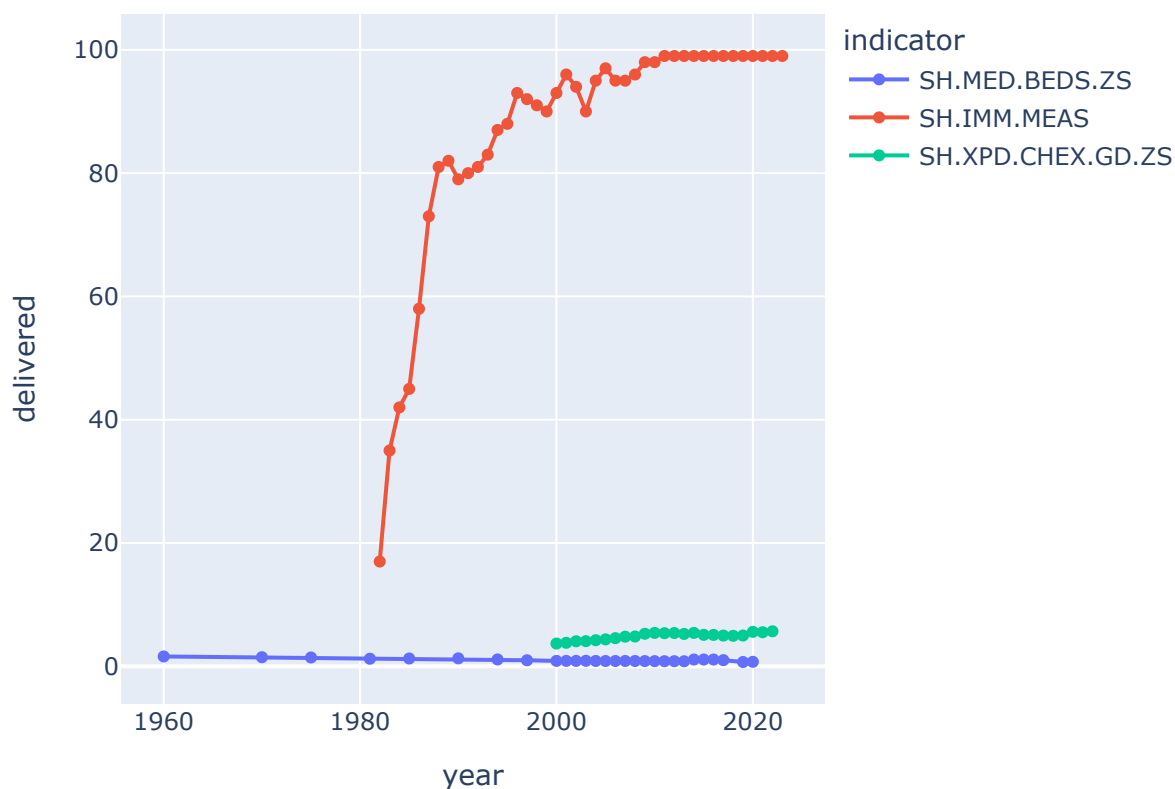
20 40 60 80 100

promised

sector source count



Health: Evidence Timeline (WDI)



Education: Evidence Timeline (WDI)

```
import pandas as pd

# 1) Final index table (subset)
idx_tbl = pd.read_csv("data/processed/sector_index.csv")
idx_tbl.to_csv("data/processed/annex_sector_index.csv", index=False)

# 2) All promises extracted (structured)
try:
    prom_all = pd.read_csv("data/interim/promises_extracted.csv")
except FileNotFoundError:
    prom_all = pd.read_csv("data/interim/promises_structured.csv")
prom_all.to_csv("data/processed/annex_promises.csv", index=False)

# 3) Evidence series (WDI)
try:
    ev = pd.read_csv("data/interim/evidence_outcomes.csv")
except FileNotFoundError:
    ev = pd.DataFrame(columns=["source", "indicator", "sector", "year", "delivered"])
```



```

ev.to_csv("data/processed/annex_evidence_outcomes.csv", index=False)

# 4) Sources catalog from earlier cells (13 + 16)
try:
    src = pd.read_csv("data/processed/sources_catalog.csv")
except FileNotFoundError:
    src = pd.DataFrame(columns=["name", "type", "topic", "url", "retrieved_at"])
src.to_csv("data/processed/annex_sources_catalog.csv", index=False)

print("Saved annex CSVs in data/processed/:")
print("- annex_sector_index.csv")
print("- annex_promises.csv")
print("- annex_evidence_outcomes.csv")
print("- annex_sources_catalog.csv")

```

22
20
18

delivered

```

Saved annex CSVs in data/processed/:
- annex_sector_index.csv
- annex_promises.csv
- annex_evidence_outcomes.csv
- annex_sources_catalog.csv

```

```

import gradio as gr
import pandas as pd
import plotly.express as px
import pathlib

def load_df():
    p = pathlib.Path("data/processed/sector_index.csv")
    return pd.read_csv(p) if p.exists() else pd.DataFrame()

def load_evidence():
    p = pathlib.Path("data/interim/evidence_outcomes.csv")
    return pd.read_csv(p) if p.exists() else pd.DataFrame()

def overview():
    df = load_df()
    if df.empty:
        return "No data yet. Run previous cells.", None
    txt = f"Overall Transparency Index: {df['index'].mean():.1f}"
    fig = px.bar(df.groupby("sector", as_index=False)["index"].mean(), x="sector",
    return txt, fig

def sector_detail(sector):
    df = load_df()
    if df.empty:
        return None, None
    sub = df[df["sector"] == sector]
    if sub.empty:
        return None, None
    fig_scatter = px.scatter(

```

```

        sub, x="promised", y="delivered",
        hover_data=["promise_text", "target_year", "delivered_year", "source"],
        title=f"{sector}: Promised vs Delivered"
    )
    ev = load_evidence()
    fig_line = None
    if not ev.empty:
        ev_sub = ev[(ev["sector"] == sector) & ev["year"].notna() & ev["delivered"]
                    if not ev_sub.empty:
                        fig_line = px.line(ev_sub.sort_values("year"), x="year", y="delivered",
                                           title=f"{sector}: Evidence Timeline")
                        fig_line.update_traces(mode="lines+markers")
    return fig_scatter, fig_line

with gr.Blocks() as demo:
    gr.Markdown("## 🌐 Civic Mirror – Transparency & Trust Index (Notebook App)")

    with gr.Tab("Overview"):
        btn = gr.Button("Refresh")
        txt = gr.Markdown()
        plot = gr.Plot()
        btn.click(overview, outputs=[txt, plot])

    with gr.Tab("Sector"):
        sector_dd = gr.Dropdown(choices=["Health", "Education", "Environment", "Gener
        btn2 = gr.Button("Show Sector")
        plot_scatter = gr.Plot()
        plot_line = gr.Plot()
        btn2.click(sector_detail, inputs=[sector_dd], outputs=[plot_scatter, plot_

demo.launch()

```

ks like you are running Gradio on a hosted Jupyter notebook, which requires a Colab notebook detected. To show errors in colab notebook, set debug=True in launching on public URL: <https://f05ae19cba88fea800.gradio.live>

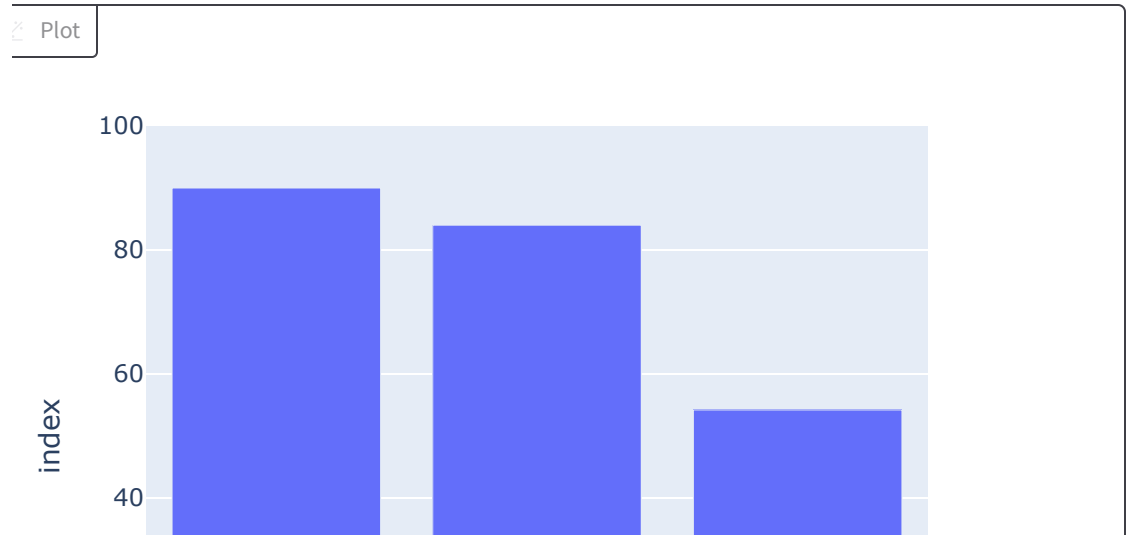
hate link expires in 1 week. For free permanent hosting and GPU upgrades, run

Civic Mirror — Transparency & Trust Index (Notebook app)

Overview Sector

Refresh

Overall Transparency Index: 76.1



```
import pandas as pd
import numpy as np

df = pd.read_csv("data/processed/sector_index.csv").copy()

# Delivery gap & on-time
df["delivery_gap"] = df["promised"] - df["delivered"]
df["on_time"] = np.where((df["delivered_year"] <= df["target_year"]), "On-time/early", "Late")

# Sector summary
summary = (df.groupby("sector", as_index=False)
            .agg(index_avg=("index", "mean"),
                 gap_avg=("delivery_gap", "mean"),
                 promises=("sector", "count"),
                 on_time_rate=("on_time", lambda s: (s=="On-time/early").mean()*100))
            .sort_values("index_avg", ascending=False))
```

```
summary.to_csv("data/processed/sector_summary.csv", index=False)
summary
```

	sector	index_avg	gap_avg	promises	on_time_rate	
0	Education	90.0	-19.473091	1	100.0	
1	Environment	84.0	NaN	1	0.0	
2	Health	54.2	14.319509	1	100.0	

Next steps:

[Generate code with summary](#)[New interactive sheet](#)

```
import requests, pandas as pd, time

HDRS = {"User-Agent": "Mozilla/5.0 (compatible; CivicMirror/1.0)"}

def wdi_latest(country_iso3, indicator):
    url = f"https://api.worldbank.org/v2/country/{country_iso3}/indicator/{indicator}"
    r = requests.get(url, headers=HDRS, timeout=60); r.raise_for_status()
    js = r.json(); data = js[1] if len(js)>1 and isinstance(js[1], list) else []
    rows = [ (int(d["date"]), d["value"]) for d in data if d.get("value") is not None ]
    return max(rows, key=lambda x: x[0]) if rows else (None, None) # (year, value)

PEERS = ["MAR", "TUN", "DZA", "EGY", "ESP", "PRT"] # tweak as you like
COMPARE = {
    "Health spend %GDP": "SH.XPD.CHEX.GD.ZS",
    "Primary enroll %": "SE.PRM.ENRR",
    "CO2 per capita": "EN.ATM.CO2E.PC",
}

rows=[]
for label, ind in COMPARE.items():
    for c in PEERS:
        try:
            y, v = wdi_latest(c, ind); time.sleep(0.12)
            if v is not None:
                rows.append({"country": c, "indicator": label, "year": y, "value": v})
        except Exception as e:
            print("skip", c, ind, e)

peer = pd.DataFrame(rows)
peer.to_csv("data/processed/peer_compare.csv", index=False)
peer.head()
```

	country	indicator	year	value	
0	MAR	Health spend %GDP	2022	5.680491	
1	TUN	Health spend %GDP	2022	6.963802	
2	DZA	Health spend %GDP	2022	3.623043	
3	EGY	Health spend %GDP	2022	4.704384	
4	ESP	Health spend %GDP	2022	9.738288	

Next steps:

[Generate code with peer](#)[New interactive sheet](#)

```
import plotly.express as px

for label in peer["indicator"].unique():
    sub = peer[peer["indicator"]==label].sort_values("value", ascending=False)
    fig = px.bar(sub, x="country", y="value", title=f"Peer comparison - {label}")
    fig.show()
```